

The Interpreter Project

CSC 413 – Software Development

Anthony J Souza

Sourced from Dr. Levine

Anthony J Souza © 2026

Computer Science
San Francisco State University

Contents

1	Introduction	4
2	The Interpreter	4
2.1	Frames (Activation Record) and the Runtime Stack	4
2.2	Heap and Dynamic Memory	6
3	Supported ByteCodes	8
3.1	Halt ByteCode	8
3.1.1	Requirements	8
3.1.2	VERBOSE mode Requirements	8
3.2	Pop ByteCode	9
3.2.1	Requirements	9
3.2.2	VERBOSE mode Requirements	9
3.3	FalseBranch ByteCode	10
3.3.1	Requirements	10
3.3.2	VERBOSE mode Requirements	10
3.4	Goto ByteCode	11
3.4.1	Requirement	11
3.4.2	VERBOSE mode Requirements	11
3.5	Store ByteCode	12
3.5.1	Requirements	12
3.5.2	VERBOSE mode Requirements	12
3.6	Load ByteCode	13
3.6.1	Requirements	13
3.6.2	VERBOSE mode Requirements	13
3.7	Lit ByteCode	14
3.7.1	Requirements	14
3.7.2	VERBOSE mode Requirements	14
3.8	Args ByteCode	15
3.8.1	Requirements	15
3.8.2	VERBOSE mode Requirements	15
3.9	Call ByteCode	16
3.9.1	Requirements	16
3.9.2	VERBOSE mode Requirements	16
3.10	Return ByteCode	17
3.10.1	Requirements	17
3.10.2	VERBOSE mode Requirements	17
3.11	Bop ByteCode	18
3.11.1	Requirements	18
3.11.2	VERBOSE mode Requirements	18
3.12	Read ByteCode	19
3.12.1	Requirements	19
3.12.2	VERBOSE mode Requirements	19

3.13	Write ByteCode	20
3.13.1	Requirements	20
3.13.2	VERBOSE mode Requirements	20
3.14	New ByteCode	21
3.14.1	Requirements	21
3.14.2	VERBOSE mode Requirements	21
3.15	HeapLoad ByteCode	22
3.15.1	Requirements	22
3.15.2	VERBOSE mode Requirements	22
3.16	HeapStore ByteCode	23
3.16.1	Requirements	23
3.16.2	VERBOSE mode Requirements	23
3.17	Free ByteCode	24
3.17.1	Requirements	24
3.17.2	VERBOSE mode Requirements	24
3.18	Label ByteCode	25
3.18.1	Requirements	25
3.18.2	VERBOSE mode Requirements	25
3.19	VERBOSE ByteCode	26
3.19.1	Requirements	26
3.19.2	VERBOSE mode Requirements	26
4	Interpreter Class	27
5	CodeTable Class	28
5.0.1	CodeTable Source Code	28
6	ByteCodeLoader Class	29
6.1	ByteCodeLoader Functions	29
7	Program Class	30
7.1	Program Class Functions	30
8	RunTimeStack Class	33
8.1	RunTimeStack Class Functions	34
9	Heap Class	36
9.1	Heap Class Skeleton	36
9.1.1	Heap Functions	37
10	Exception Packages	39
11	VirtualMachine Class	40
12	VERBOSE mode	41
12.1	VERBOSE Implementation Notes	43

13 Coding Hints	44
13.1 Possible Order of Implementation	44
13.2 Other Coding Hints	44
14 Setup	46
14.1 Importing Project	46
15 Requirements	46
16 Submission	48

1 Introduction

For this assignment you will be implementing an interpreter for the mock language X. You can think of the mock language X as a simplified version of Java. The interpreter is responsible for processing bytecodes that are created from source code files with the extension `.x`. The interpreter and the VirtualMachine (this will be implemented by you as well) will work together to run a program written in the language X. The two sample programs are a recursive version of computing the nth Fibonacci number and finding the factorial of a number. These compiled bytecode files have the extension `.x.cod`.

And as always, if there are any questions please ask them in discord AND in class. This promotes collaborative thinking which is important. NOTE THIS IS AN INDIVIDUAL ASSIGNMENT but you can collaborate with other students.

2 The Interpreter

In the following sections you will find coding hints, class requirements, and other information to help you implement and complete the interpreter project. Please read these pages a few times before you begin coding.

2.1 Frames (Activation Record) and the Runtime Stack

Frames or activation records are simply the set of variables (actual arguments, local variables, and temporary storage) for each function called during the execution of a program. Given that function calls and returns happen in a LIFO fashion, we will use a Stack to hold and maintain our frames.

For example, if we have the following execution sequence:

<pre>Main Call A from Main Call B from A</pre>
--

With the sequence above, we will get the following runtime stack (Figure 1):

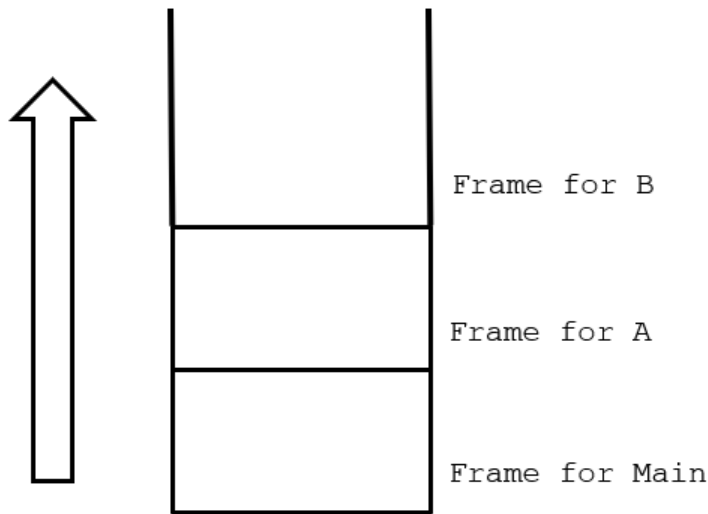


Figure 1: View of runtime stack as methods are called.

Now if we take this concept and apply it to some code written in the Language X we can see a better example of our runtime stack.

```

program { int i int j
    int f ( int i ) { 2
        int j int k 3
        return i + j + k + 2 4
    }
int m 1
m = f(3) 5
i = write(j+m)
}

```

Figure 2: Small code sample in the Language X.

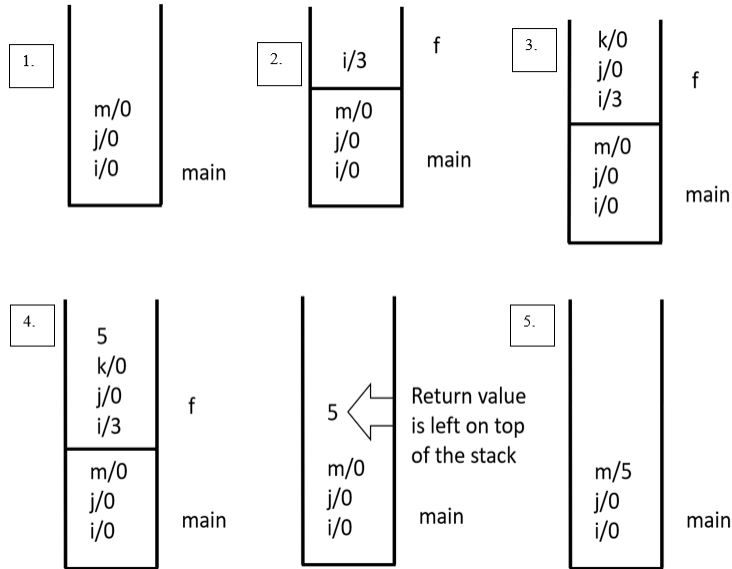


Figure 3: View of Runtime Stack as code is executed.

In the figure above we can see what happens as a program in the X language is executed. Below is a general idea of what is going on in the virtual machine.

- At Bubble 1 – When code reaches this point, the virtual machine has pushed 3 values: `i`, `j`, and `k`. All three have the values of 0.
- At Bubble 2 – When we reach this bubble, we have done a few things. First we need to load the literal value of 3 on the runtime stack. This will act as an argument for the function `f`. Then we create a new activation frame for function `f`, which includes the top value of the runtime stack since it takes one argument. Then we can call function `f`.
- At Bubble 3 – After executing up to bubble 3, the virtual machine has pushed 2 more values to the runtime stack. This is to declare and initialize the variables `j` and `k`. NOTE: `j` in `main`'s activation frame and `j` in `f`'s activation frame DO NOT conflict.
- At Bubble 4 – After executing up to bubble 4, the virtual machine has evaluated the expression `i + j + k + 2` and pushed the result onto the top of the stack.
- At Bubble 5 – Once the virtual machine gets to bubble 5, it has done a few things. First, it saves the value 5 (the return value) and then clears the rest of function `f`'s activation record. Then it removes the frame boundary for function `f`. Then it pushes the return value back to the runtime stack. Then it moves the value 5 into the slot on the runtime stack that stores the variable `m`.

2.2 Heap and Dynamic Memory

The `RunTimeStack` stores activation frames, local variables, function arguments, and temporary expression values. Dynamic memory should be stored in a separate Heap owned

by the VirtualMachine. The Heap is useful for values that should live outside of a single activation frame, such as array-like blocks of integers.

The Heap uses a fixed maximum size of 2 MB. You should treat each integer value stored in the heap as using 4 bytes. A heap allocation request must fail if it would cause the Heap to exceed this maximum size.

Heap references are represented as integer addresses. These addresses may be pushed to the RunTimeStack, stored in local variables, passed as function arguments, and returned from functions. The Heap itself should not reach into the RunTimeStack. Instead, heap ByteCodes should request operations from the VirtualMachine, and the VirtualMachine should coordinate moving values between the RunTimeStack and the Heap.

For example, if a heap block stores the values 5 and 7, the values should be loaded onto the RunTimeStack before using BOP to add them:

```
LOAD 0 arr
LIT 0
HLOAD
LOAD 0 arr
LIT 1
HLOAD
BOP +
```

The Heap must perform bounds checking and address validation. Accessing an invalid address, accessing outside the allocated block, allocating more than the maximum heap size, or using a freed heap block should produce a heap-related exception.

3 Supported ByteCodes

Below you will find the descriptions of the ByteCodes your interpreter must support. Each ByteCode must be implemented to the correct specifications outlined below. It is important that you understand what each ByteCode does as this will help you implement the ByteCodes.

3.1 Halt ByteCode

The HALT ByteCode is used to alert the virtual machine that program execution is to be stopped. Halt is not allowed to kill the interpreter program. Therefore, Halt may not call `System.exit` to stop the execution of the program.

3.1.1 Requirements

- Request that the VirtualMachine halt execution of current program.
- Halt takes no arguments.
- Halt ByteCode does not need to be displayed when VERBOSE mode is on.
- Halt cannot execute a `system.exit` function call.

3.1.2 VERBOSE mode Requirements

The Halt bytecode does not need to be displayed when VERBOSE mode is on.

3.2 Pop ByteCode

The Pop ByteCode will be used to remove values from the runtime stack. The Pop ByteCode is not allowed to remove values across frame boundaries. It is the implementer's responsibility to ensure that Pop is only allowed to pop the appropriate number of values. Pop has one argument, N, which is the number of values to be popped. You cannot make any assumptions about the value of N other than it is a strictly positive number.

3.2.1 Requirements

- Pop takes one argument which is the number of values to remove from the run time stack.
- Pop is not allowed to operate across frame boundaries.
- If VERBOSE mode is on, Pop is required to be displayed every time it executes.
- The Pop bytecode cannot detect when VERBOSE mode is on nor should it ask the VirtualMachine if VERBOSE mode is on.

3.2.2 VERBOSE mode Requirements

The Pop bytecode has the following display syntax when VERBOSE mode is on:

POP N

where N is the number of times pop is requesting to be executed. Note, this value can be different than the actual number of POPs executed depending on current frame size.

3.3 FalseBranch ByteCode

The FalseBranch ByteCode will be used to execute conditional jumps (think of executing control structures like if-statements and loops). FalseBranch will have one argument. This argument is a Label that will mark a place in the program to jump to. FalseBranch will remove the top value of the runtime stack and check to see if the value is 0. If the value is 0, jump to the corresponding label. If the value is something else, move to the next ByteCode in the program. FalseBranch will need to have its label address calculated before the program begins executing. This requires finding where the destination of the jump is numerically located in the program.

3.3.1 Requirements

- FalseBranch takes one argument, a label to jump to.
- FalseBranch's label address will need to be resolved. This requires computing where FalseBranch will jump to if the value popped from the stack is 0. Address resolution needs to be done before you begin executing the program. This will be discussed later in this document.
- Remove the top value of the stack.
 - if value is 0, jump to label.
 - if value is not 0, move to next ByteCode. (do nothing)
- If VERBOSE mode is on, FalseBranch ByteCode is required to be displayed after each execution.
- The FalseBranch bytecode cannot detect when VERBOSE mode is on nor should it ask the VirtualMachine if VERBOSE mode is on.

3.3.2 VERBOSE mode Requirements

The FalseBranch bytecode has the following VERBOSE syntax:

```
FALSEBRANCH label
```

where label is the original label read from the .cod files.

3.4 Goto ByteCode

The Goto ByteCode is used to jump to Labels in our programs. Goto is considered an unconditional jump. This means regardless of the state of the program, we take the jump. Goto has one argument, the label it needs to jump to. Like FalseBranch, Goto's label needs to go through address resolution as well.

3.4.1 Requirement

- Goto has one argument, a label to jump to.
- Goto's Label must have its address resolved before the program begins executing. More on this later.
- If VERBOSE mode is on, Goto is required to be displayed after each execution.
- The Goto bytecode cannot detect when VERBOSE mode is on nor should it ask the VirtualMachine if VERBOSE mode is on.

3.4.2 VERBOSE mode Requirements

The Goto bytecode has the following display format in VERBOSE mode:

```
GOTO label
```

where label is the original label read from the .cod files.

3.5 Store ByteCode

The Store ByteCode will be used to move values from the top of the runtime stack to an offset in the current frame. This offset starts from the beginning of the frame. The idea behind this ByteCode is that it is needed to do operations like assignments. The Store ByteCode is not allowed to operate across frame boundaries.

3.5.1 Requirements

- The Store ByteCode can have 1 to 2 arguments.
 - one argument is the offset in the current frame where the value that is popped is to be stored.
 - The second argument, if present, is the identifier (variable) the value being moved belongs to. This will be used for VERBOSE mode.
- Store must pop the top of the runtime stack and store the value at the offset in the current frame.
- Store cannot operate across frame boundaries.
- Store's offset must be non-negative.
- If VERBOSE mode is on, Store is required to be displayed after each execution.
- The Store bytecode cannot detect when VERBOSE mode is on nor should it ask the VirtualMachine if VERBOSE mode is on.

3.5.2 VERBOSE mode Requirements

The Store bytecode has the following display format:

```
Basic Syntax : STORE <offset> [<id> <id>=<value-popped>]
<offset> is the index in the current frame to store into
<id> is a variable identifier [optional]
Example      :
If we assume the RunTimeStack has the following values
BEFORE executing the Store bytecode:
[0,1,2,3]
Then executing a Store 1 k would produce:
STORE 1 k    k=3
```

Note that if id is missing or not supplied, then your toString should only print

```
STORE 1
```

3.6 Load ByteCode

The Load ByteCode will be used to move values from an offset in the current frame to the top of the stack. This offset works from the beginning of the frame. The purpose behind this ByteCode is that it is needed to set up copies of values for things like expressions and arguments for functions. The Load ByteCode is not allowed to operate across frame boundaries.

3.6.1 Requirements

- The Load ByteCode can have 1 to 2 arguments.
 - one argument is the offset in the current frame where the value is to be copied from.
 - The second argument, if present, is the identifier (variable) the value belongs to. This will be used for VERBOSE mode.
- Load must copy the value at the offset in the current frame and push it to the top of the stack.
- Load must not remove any values from the runtime stack.
- Load cannot operate across frame boundaries.
- Load's offset must be non-negative.
- It is not required that the identifier argument exists.
- If VERBOSE mode is on, Load is required to be displayed after each execution.
- The Load bytecode cannot detect when VERBOSE mode is on nor should it ask the VirtualMachine if VERBOSE mode is on.

3.6.2 VERBOSE mode Requirements

```
Basic Syntax : LOAD <offset> [<id> <load id>]  
<offset> is the index in the current frame to load from  
<id> is a variable identifier  
Example      : LOAD 2 j      <load j>
```

If id is missing, then Load's toString will return just:

```
LOAD offset
```

Therefore, using the above example, we have:

```
LOAD 2
```

3.7 Lit ByteCode

The Lit ByteCode is used to push literal values to the runtime stack. In some cases, Lit ByteCodes will be accompanied by an id (a variable name). This id represents the variable name the value belongs to. This id is optional.

3.7.1 Requirements

- The Lit ByteCode takes 1 or 2 arguments.
- The Lit ByteCode must push a value to the RunTimeStack.
- It is not required that the identifier argument exists.
- If VERBOSE mode is on, Lit ByteCode is required to be displayed after each execution.
- The Lit bytecode cannot detect when VERBOSE mode is on nor should it ask the VirtualMachine if VERBOSE mode is on.

3.7.2 VERBOSE mode Requirements

Basic Syntax : `LIT <value> [<id> int <id>]`
`<value>` is the value being pushed to RunTimeStack
`<id>` is a variable identifier [optional]

If we were to execute a `LIT 0 j` then Lit's toString will return:

```
LIT 0 j    int j
```

If the id is missing, then Lit has the following display format:

```
LIT 0
```

3.8 Args ByteCode

The Args ByteCode is used to set up how many arguments a function has. The Args ByteCode will always be executed just before a Call ByteCode. The Args ByteCode has one argument, the number of values that are arguments for the next function call. This value, N, will be used to determine how many values starting from the top of the runtime stack will be a part of a newly created activation frame for the next function call. Args will need to figure out where in the runtime stack this new frame begins and push this index into the framePointer stack.

3.8.1 Requirements

- The Args bytecode has one argument, the number of values that will be a part of the new activation frame.
- The Args ByteCode will need to push the starting index of the new frame to the framePointer stack.
- If VERBOSE mode is on, the Args ByteCode is required to be displayed after each execution.
- The Args bytecode cannot detect when VERBOSE mode is on nor should it ask the VirtualMachine if VERBOSE mode is on.

3.8.2 VERBOSE mode Requirements

The Args bytecode has the following display syntax:

ARGS n

where n is the number of args of the new call frame.

3.9 Call ByteCode

The Call ByteCode is what the VirtualMachine uses to jump to locations in the program to execute sections of code we call functions. When encountered, the Call ByteCode will jump to the corresponding label in the program. The ByteCode is also responsible for keeping track of where control should return to when a function completes its execution.

3.9.1 Requirements

- Call ByteCode takes 1 argument, a label to jump to.
- Call Code must go through address resolution to figure out where it needs to jump to in the Program before the program is run.
- Call Code must store a return address onto the Return Address Stack.
- Call Code must jump to the address in the program that corresponds to a label code (this address is computed during address resolution).
- If VERBOSE mode is on, the Call ByteCode is required to be displayed after each execution.
- The Call bytecode cannot detect when VERBOSE mode is on nor should it ask the VirtualMachine if VERBOSE mode is on.

3.9.2 VERBOSE mode Requirements

The Call bytecode has the following display format:

```
Basic format : CALL <id>    <base-id>(<args>)
<id> is a function identifier
<base-id> is the actual id of the function;
e.g. for CALL f<<2>>, the <base-id> is f
<args> are the function arguments.
Example      :
If we assume the RunTimeStack has the following values
    [0,1,2] [3,4,5]
And we execute a CALL f<<3>>.
Before the CALL code is executed, an ARGS 3 has been executed.
Then the display of the call code should look as follows:
CALL f<<3>>    f(3,4,5)
```

3.10 Return ByteCode

The Return ByteCode will be used to return from functions and to put return values in the correct position on the runtime stack. The interpreter project will use this convention for handling arguments and return values. Callers of functions are required to set up arguments for the functions they call. Functions themselves (callees) are required to put return values in the correct spot just before returning from a function. Note that this is a convention we will adhere to and is something that is not enforced programmatically. This means if you fail to follow this convention, transient bugs can happen. The Return ByteCode has a lot of responsibility. The steps for completing a return are important.

3.10.1 Requirements

- The Return ByteCode can take 0 to 1 arguments. The arguments have no effect on its functionality but do affect the VERBOSE mode display of the ByteCode.
- The Return ByteCode must store the return value at the top of the runtime stack.
- The Return ByteCode must empty the current frame of all values when the function is complete.
- The Return ByteCode must pop the top value from the framePointer stack to remove the frame boundary.
- The Return ByteCode must pop the top of the return address stack and save it into the program counter.
- If VERBOSE mode is on, the Return ByteCode is required to be displayed after each execution.
- The Return bytecode cannot detect when VERBOSE mode is on nor should it ask the VirtualMachine if VERBOSE mode is on.

3.10.2 VERBOSE mode Requirements

The Return bytecode has the following display syntax:

```
Basic Syntax : RETURN <id>      EXIT <base-id>:<value>
<id> is a function identifier
<value> is the value being returned from the function
<base-id> is the actual id of the function;
e.g. for RETURN f<<2>>, the <base-id> is f
Example      : RETURN f<<2>>  EXIT f : 4
```

3.11 Bop ByteCode

The Bop ByteCode is used to implement binary operations for the Interpreter Project. The Bop ByteCode will need to remove 2 values from the runtime stack and operate on them accordingly. The result needs to be pushed back to the top of the stack. Be careful, the order of the operands matters.

3.11.1 Requirements

- Bop must pop 2 values from the runtime stack.
- Bop must push 1 value, the result, back to the top of the runtime stack.
- Bop must implement the following binary operations:

+-----+-----+			
Addition	: +	Subtraction	: -
Division	: /	Multiplication	: *
Equality	: ==	Not Equal	: !=
Less Than Equal	: <=	Less Than	: <
Greater Than Equal	: >=	Greater Than	: >
Logical And	: &	Logical OR	:
+-----+-----+			

- If VERBOSE mode is on, the Bop ByteCode is required to be displayed after each execution.
- The Bop bytecode cannot detect when VERBOSE mode is on nor should it ask the VirtualMachine if VERBOSE mode is on.

3.11.2 VERBOSE mode Requirements

The Bop bytecode has the following display syntax:

BOP operator

where operator is the operation executed. For example, if we execute a + operation, displaying the bytecode in VERBOSE mode would be:

BOP +

3.12 Read ByteCode

The Read ByteCode is used to read user input from the keyboard. Only integers should be accepted from users. You may use Scanners or BufferedReaders to read input from the user.

3.12.1 Requirements

- When asking for user input, use the following prompt: "Please enter an integer : "
- The Read ByteCode needs to verify that the value given is actually a number. If an invalid number is given, state that the input is invalid and ask for another value. Continue to do so until a valid value is given.
- Push the validated integer to the VirtualMachine's RunTimeStack.
- If VERBOSE mode is on, the Read bytecode is required to be displayed after each execution.
- The Read bytecode cannot detect when VERBOSE mode is on nor should it ask the VirtualMachine if VERBOSE mode is on.

3.12.2 VERBOSE mode Requirements

The Read bytecode has the following display syntax:

READ

3.13 Write ByteCode

The Write ByteCode is used to display information to the console. The only thing Write is allowed to display is the top value of the runtime stack. No other information is allowed to be shown.

3.13.1 Requirements

- Prints the top of the runtime stack to the console.
- NO OTHER information can be printed by the Write ByteCode when printing the value.
- If VERBOSE mode is on, Write is required to be displayed after each execution.
- The Write bytecode cannot detect when VERBOSE mode is on nor should it ask the VirtualMachine if VERBOSE mode is on.

3.13.2 VERBOSE mode Requirements

The Write bytecode has the following display syntax:

WRITE

3.14 New ByteCode

The New ByteCode is used to allocate a new block of integer values on the Heap. The size argument represents the number of integer slots requested. After allocation, the Heap returns an integer address that is pushed onto the RunTimeStack. This address is the reference that future heap ByteCodes will use.

3.14.1 Requirements

- New takes one argument, N, which is the number of integer slots to allocate.
- N must be strictly positive.
- New must request that the VirtualMachine allocate a heap block.
- New must push the returned heap address onto the RunTimeStack.
- New must not directly access the Heap object.
- If allocation would exceed the maximum heap size of 2 MB, the heap operation should throw a HeapOutOfMemoryException.
- If VERBOSE mode is on, New is required to be displayed after each execution.
- The New bytecode cannot detect when VERBOSE mode is on nor should it ask the VirtualMachine if VERBOSE mode is on.

3.14.2 VERBOSE mode Requirements

The New bytecode has the following display syntax:

NEW N

where N is the number of integer slots requested.

3.15 HeapLoad ByteCode

The HeapLoad ByteCode is used to copy a value from the Heap to the top of the RunTimeStack. HeapLoad does not take arguments from the .cod line. Instead, it uses values already on the RunTimeStack. HeapLoad will pop an index and a heap address, load the value at that heap location, and push the loaded value onto the RunTimeStack.

3.15.1 Requirements

- HeapLoad takes no arguments from the .cod file.
- HeapLoad must pop the heap index from the top of the RunTimeStack.
- HeapLoad must pop the heap address from the RunTimeStack after popping the index.
- HeapLoad must request that the VirtualMachine load the value stored at the given heap address and index.
- HeapLoad must push the loaded value onto the RunTimeStack.
- HeapLoad must not directly access the Heap object.
- HeapLoad must reject invalid heap addresses.
- HeapLoad must reject indexes that are negative or outside the allocated heap block.
- HeapLoad must reject access to heap blocks that have been freed.
- If VERBOSE mode is on, HeapLoad is required to be displayed after each execution.
- The HeapLoad bytecode cannot detect when VERBOSE mode is on nor should it ask the VirtualMachine if VERBOSE mode is on.

3.15.2 VERBOSE mode Requirements

The HeapLoad bytecode has the following display syntax:

HLOAD

For example, if the RunTimeStack contains a heap address followed by index 0, HLOAD should replace those two values with the heap value loaded from that address and index.

3.16 HeapStore ByteCode

The HeapStore ByteCode is used to move a value from the RunTimeStack into the Heap. HeapStore does not take arguments from the .cod line. Instead, it uses values already on the RunTimeStack. HeapStore will pop a value, an index, and a heap address. The popped value is stored at the given heap address and index.

3.16.1 Requirements

- HeapStore takes no arguments from the .cod file.
- HeapStore must pop the value to store from the top of the RunTimeStack.
- HeapStore must pop the heap index from the RunTimeStack after popping the value.
- HeapStore must pop the heap address from the RunTimeStack after popping the index.
- HeapStore must request that the VirtualMachine store the value at the given heap address and index.
- HeapStore must not directly access the Heap object.
- HeapStore must reject invalid heap addresses.
- HeapStore must reject indexes that are negative or outside the allocated heap block.
- HeapStore must reject access to heap blocks that have been freed.
- If VERBOSE mode is on, HeapStore is required to be displayed after each execution.
- The HeapStore bytecode cannot detect when VERBOSE mode is on nor should it ask the VirtualMachine if VERBOSE mode is on.

3.16.2 VERBOSE mode Requirements

The HeapStore bytecode has the following display syntax:

HSTORE

For example, if the RunTimeStack contains a heap address, index 0, and value 42, HSTORE should store 42 at index 0 of the heap block.

3.17 Free ByteCode

The Free ByteCode is used to release a heap block that was previously allocated by New. Free takes no arguments from the .cod line. It pops a heap address from the RunTimeStack and requests that the VirtualMachine free that address.

3.17.1 Requirements

- Free takes no arguments from the .cod file.
- Free must pop a heap address from the RunTimeStack.
- Free must request that the VirtualMachine free the heap block at that address.
- Free must not directly access the Heap object.
- Free must reject invalid heap addresses.
- Free must reject freeing the same heap block more than once.
- Any later access to a freed heap block must be rejected.
- If VERBOSE mode is on, Free is required to be displayed after each execution.
- The Free bytecode cannot detect when VERBOSE mode is on nor should it ask the VirtualMachine if VERBOSE mode is on.

3.17.2 VERBOSE mode Requirements

The Free bytecode has the following display syntax:

FREE

3.18 Label ByteCode

The Label bytecode is a bytecode that has no functionality. Its sole purpose is to mark locations in the program where other bytecodes can jump to. Label bytecodes will be used in address resolution so other ByteCodes know where they are supposed to jump to.

3.18.1 Requirements

- Label takes one argument, a label which is used to denote a location in the program.
- If VERBOSE mode is on, displaying the Label bytecode is **optional**.

3.18.2 VERBOSE mode Requirements

If you decide to display the Label bytecode during VERBOSE mode, it should have the following display format:

```
LABEL label
```

where label is the string value that is the label of where to jump to.

3.19 VERBOSE ByteCode

The VERBOSE ByteCode is used to turn VERBOSE mode ON and OFF. VERBOSE mode in the interpreter is only turned on when VERBOSE mode is set to ON.

3.19.1 Requirements

- The VERBOSE ByteCode has 1 argument: either "ON" or "OFF".
- The VERBOSE ByteCode must request the VirtualMachine to turn VERBOSE mode either "ON" or "OFF".
- The VERBOSE ByteCode is to be displayed after turning VERBOSE mode ON or OFF.

3.19.2 VERBOSE mode Requirements

The VERBOSE ByteCode has the following display format:

VERBOSE value

where value is either ON or OFF, must be in all capital letters as well.

4 Interpreter Class

The interpreter class is used as the entry point for this project. It is responsible for the following:

1. Reading in the .cod file to be run from the command line.
2. Initialize the CodeTable.
3. Calling loadCodes to load the program.
4. Calling resolve address for the built program.
5. Creating the VirtualMachine and beginning execution of the program.

You can view the source code of the Interpreter class by looking in the Interpreter.java file in the project repository.

When running the program, make sure to let your IDE/Text Editor know what program it is running by supplying a command line argument in the configuration settings.

This file cannot be modified

5 CodeTable Class

The CodeTable class is used to store a mapping of ByteCodes as they appear in the .cod files to their class names in the project. Some skeleton code is provided. You are required to complete the implementation of the CodeTable. Be very careful that you use the correct values for the key/value pairs in the mapping. If any value is incorrect, the interpreter will not work.

These tokens should map to the concrete ByteCode classes that implement heap allocation, heap loading, heap storing, and heap freeing.

5.0.1 CodeTable Source Code

```
1 package interpreter.loaders;
2
3 /**
4  * Code table of byte codes in language X.
5  *
6  * Maps a bytecode token to the class name that implements it.
7  */
8 public final class CodeTable {
9
10     private CodeTable() {
11         // do nothing
12     }
13
14     /**
15      * Fills code table with class name mappings.
16      */
17     public static void init() {
18
19     }
20
21     /**
22      * Returns the ByteCode class name for a given token.
23      *
24      * @param token bytecode to map. For example, HALT -->
25      *           HaltCode
26      * @return class name of bytecode
27      */
28     public static String getClassName(String token) {
29         return null;
30     }
31 }
```

6 ByteCodeLoader Class

The ByteCodeLoader class is responsible for loading bytecodes from the source code file into a data structure that stores the entire program. We will use an ArrayList to store our bytecodes. This ArrayList will be contained inside of a Program object. Adding and getting ByteCodes will go through the Program class.

The ByteCodeLoader class will implement a loadCodes function that does the following:

1. Opens a file reader to parse the source code.
2. For each line in the source code, does the following:
 1. Tokenizes the current line read.
 2. With the first token on each line, instantiates a ByteCode instance.
 3. If there are more tokens, moves them to an ArrayList of strings and passes these arguments to the bytecode's init function.
 4. Stores the fully initialized ByteCode instance into the program data structure.
3. returns program object.

6.1 ByteCodeLoader Functions

```
1 /**
2  * Constructs ByteCodeLoader object given a COD source code
3  * file name
4  * @param fileName name of .cod File to load.
5  */
6 public ByteCodeLoader(String fileName){
7     this.codSourceFileName = fileName;
8 }
9
10 /**
11  * Loads a program from a .cod file.
12  * @return a constructed Program Object.
13  * @throws InvalidProgramException exception thrown when
14  * loadCodes fails.
15  */
16 public Program loadCodes() throws InvalidProgramException {
17     return null;
18 }
```

7 Program Class

The Program class will be responsible for storing all the ByteCodes read from the source file. We will store ByteCodes in an ArrayList which has a designated type of ByteCode. This will ensure only ByteCodes and subclasses of ByteCode can be added to the ArrayList.

7.1 Program Class Functions

```
1 package interpreter.loaders;
2
3 import java.util.ArrayList;
4 import java.util.List;
5
6 public class Program {
7
8     private List<ByteCode> program;
9
10    /**
11     * Instantiates a program object using an
12     * ArrayList
13     */
14    public Program() {
15
16    }
17
18    /**
19     * Gets the size of the current program.
20     *
21     * @return size of program
22     */
23    public int getSize() {
24        return 0;
25    }
26
27    /**
28     * Grabs an instance of a bytecode at an index.
29     *
30     * @param programCounter index of bytecode to get.
31     * @return a bytecode.
32     */
33    public ByteCode getCode(int programCounter) {
34        return null;
35    }
36
37    /**
```

```

38     * Adds a bytecode instance to the Program List.
39     *
40     * @param c bytecode to be added
41     */
42     public void addCode(ByteCode c) {
43
44     }
45
46     /**
47     * Makes multiple passes through the program ArrayList
48     * resolving addresses for the Goto, Call, and FalseBranch
49     * bytecodes. These bytecodes can only jump to Label
50     * codes that have a matching label value.
51     * HINT: make note of what type of data-structure
52     * ByteCodes are stored in.
53     * **** METHOD SIGNATURE CANNOT BE CHANGED ****
54     */
55     public void resolveAddress() {
56
57     }
58 }

```

Resolving symbolic addresses can be done in many ways. It is up to you to design a clean implementation for mapping the generated labels the compiler uses to absolute addresses in the program. An example is given below.

The Program class will hold the ByteCode program loaded from the file. It will also resolve symbolic addresses in the program. For example, if we have the following program below

```
0. FALSEBRANCH continue<<6>>
1. LIT 2
2. LIT 2
3. BOP ==
4. FALSEBRANCH continue<<9>>
5. LIT 1
6. ARGS 1
7. CALL Write
8. STORE 0 i
9. LABEL continue<<9>>
10. LABEL continue<<6>>
```

After address resolution has been completed the program should look like the following (NOTE you should not modify the original source code file, these changes are made to the Program object):

```
0. FALSEBRANCH 10
1. LIT 2
2. LIT 2
3. BOP ==
4. FALSEBRANCH 9
5. LIT 1
6. ARGS 1
7. CALL Write
8. STORE 0 i
9. LABEL continue<<9>>
10. LABEL continue<<6>>
```

8 RunTimeStack Class

Records and processes the stack of active frames. This class will contain two data structures used to help the VirtualMachine execute the program. It is **VERY** important that you do not return any of these data structures. This means there should be NO getters and setters for these data structures. These data structures must remain private as well.

The RunTimeStack class will use the following two data structures:

```
1 // This stack is used to record the beginning of
2 // each activation record(frame) when calling functions.
3 private Stack<Integer> framePointer;
4
5 // This ArrayList is used to represent the runtime stack.
6 // It will be an ArrayList because we will need to access
7 // ALL locations of the runtime stack.
8 private ArrayList<Integer> runTimeStack;
```

When trying to understand the RunTimeStack class, you should always use BOTH data structures. Both are needed for the correct execution of the program.

Recall from earlier:

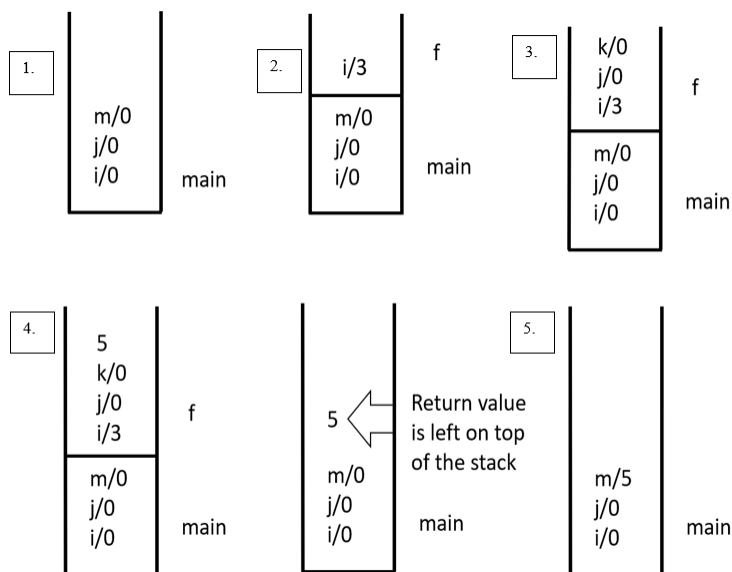


Figure 4: View of Runtime Stack as code is executed.

Initially the framePointer stack will have 0 for step 1 above. When we call function f at step 2, framePointer stack will have the values 0 and 3 since 0 is the start of main and 3 is the start of f. At step 4.1 we pop the framePointer stack (3 is popped). Then the framePointer stack will only contain the value 0. This is Main's starting frame index. Note that when

transitioning from step 4 to 4.1 there is more work needed to clean up each frame before returning from functions.

8.1 RunTimeStack Class Functions

```
1 /**
2  * Used for displaying the current state of the runTimeStack.
3  * It will print portions of the stack based on respective
4  * frame markers.
5  * Example [1,2,3] [4,5,6] [7,8]
6  * Frame pointers would be 0,3,6
7  */
8 public String verboseDisplay() { }
9
10 /**
11  * returns the top of the runtime stack, but does not remove
12  * @return copy of the top of the stack.
13  */
14 public int peek(){ }
15
16 /**
17  * push the value i to the top of the stack.
18  * @param i value to be pushed.
19  * @return value pushed
20  */
21 public int push(int i) { }
22
23 /**
24  * removes to the top of the runtime stack.
25  * @return the value popped.
26  */
27 public int pop() { }
28 /**
29  * Takes the top item of the run time stack, and stores
30  * it into a offset starting from the current frame.
31  * @param offsetInFrame number of slots above current frame
32  * marker
33  * @return the item just stored
34  */
35 public int store(int offsetInFrame) { }
36 /**
37  * Takes a value from the run time stack that is at offset
38  * from the current frame marker and pushes it onto the top of
39  * the stack.
```

```
39  * @param offsetInFrame number of slots above current frame
    marker
40  * @return item just loaded into the offset
41  */
42 public int load(int offsetInFrame) { }
43
44 /**
45  * create a new frame pointer at the index offset slots down
46  * from the top of the runtime stack.
47  * @param offsetFromTopOfRunStack slots down from the top of
    the runtime stack
48  */
49 public void newFrameAt(int offsetFromTopOfRunStack) { }
50
51 /**
52  * pop the current frame off the runtime stack. Also removes
53  * the frame pointer value from the FramePointer Stack.
54  */
55 public void popFrame() { }
```

9 Heap Class

The Heap class records and processes dynamically allocated memory for the VirtualMachine. Heap memory is separate from the RunTimeStack. The RunTimeStack stores values, activation frames, and temporary expression results. The Heap stores allocated blocks that can be accessed by heap addresses.

The Heap class should be placed in the `interpreter.virtualmachine` package. The VirtualMachine owns the Heap and ByteCodes should request heap operations through the VirtualMachine. ByteCodes should not directly access the Heap object.

9.1 Heap Class Skeleton

```
1 package interpreter.virtualmachine;
2
3 import java.util.Map;
4 import java.util.Set;
5
6 class Heap {
7
8     private static final int BYTES_PER_MEGABYTE = 1024 * 1024;
9     private static final int MAX_HEAP_SIZE_BYTES = 2 *
10         BYTES_PER_MEGABYTE;
11     private static final int MAX_HEAP_SLOTS =
12         MAX_HEAP_SIZE_BYTES / Integer.BYTES;
13
14     private final Map<Integer, int[]> allocations;
15     private final Set<Integer> freed;
16     private int nextAddress;
17     private int usedSlots;
18
19     public Heap() {
20     }
```

The Heap implementation will need to track allocated blocks, used heap space, and whether a block has been freed. You may choose the internal data structures, but they must remain private. There should be no getters that expose the Heap's internal storage.

The Heap must validate all heap operations:

- Allocation sizes must be strictly positive.
- Allocation must not exceed 2 MB.
- Heap addresses must refer to valid allocated blocks.

- Heap indexes must be within the bounds of the allocated block.
- Freed heap blocks must not be accessed again.

9.1.1 Heap Functions

```

1 /**
2  * Allocates a contiguous block of size integer slots on the
   heap.
3  *
4  * @param size the number of integer slots to allocate
5  * @return the base address of the newly allocated block
6  * @throws HeapOutOfMemoryException if the allocation would
   exceed the maximum heap size
7  */
8 int allocate(int size) throws HeapOutOfMemoryException { }
9
10 /**
11  * Loads the integer value stored at address + offset on the
   heap.
12  *
13  * @param address the base address of the allocated block
14  * @param offset the index within the block to read from
15  * @return the integer value at the specified location
16  * @throws InvalidHeapAddressException if address was never
   allocated
17  * @throws HeapUseAfterFreeException if the block at
   address has been freed
18  * @throws HeapBoundsException if offset is out of
   range for the block
19  */
20 int load(int address, int offset)
21     throws InvalidHeapAddressException,
22         HeapUseAfterFreeException, HeapBoundsException {
23     return -1;
24 }
25
26 /**
27  * Stores value at address + offset on the heap.
28  *
29  * @param address the base address of the allocated block
30  * @param offset the index within the block to write to
31  * @param value the integer value to store
32  * @throws InvalidHeapAddressException if address was never
   allocated

```

```

32 * @throws HeapUseAfterFreeException    if the block at
    address}has been freed
33 * @throws HeapBoundsException          if offset is out of
    range for the block
34 */
35 void store(int address, int offset, int value)
36     throws InvalidHeapAddressException,
    HeapUseAfterFreeException, HeapBoundsException { }
37
38 /**
39 * Frees the allocated block at address, making its slots
    available again.
40 *
41 * @param address the base address of the block to free
42 * @throws InvalidHeapAddressException if address was never
    allocated
43 * @throws HeapUseAfterFreeException    if the block at
    address has already been freed
44 */
45 void free(int address) throws InvalidHeapAddressException,
    HeapUseAfterFreeException { }
46
47 /**
48 * Validates that address refers to a live (non-freed)
    allocation and that
49 * offset falls within the bounds of that allocation.
50 *
51 * @param address the base address to validate
52 * @param offset  the offset within the block to validate
53 * @throws InvalidHeapAddressException if address was never
    allocated
54 * @throws HeapUseAfterFreeException    if the block at
    address has been freed
55 * @throws HeapBoundsException          if offset is out of
    range for the block
56 */
57 private void checkAccess(int address, int offset) throws
    InvalidHeapAddressException, HeapUseAfterFreeException,
    HeapBoundsException { }

```

10 Exception Packages

Exceptions should be organized by the part of the interpreter that owns the error.

The `InvalidProgramException` belongs in the `interpreter.loaders.exceptions` package:

```
interpreter/loaders/exceptions/InvalidProgramException.java
```

Heap runtime exceptions belong in the `interpreter.virtualmachine.exceptions` package:

```
interpreter/virtualmachine/exceptions/HeapOutOfMemoryException.java
interpreter/virtualmachine/exceptions/InvalidHeapAddressException.java
interpreter/virtualmachine/exceptions/HeapBoundsException.java
interpreter/virtualmachine/exceptions/HeapUseAfterFreeException.java
```

`HeapOutOfMemoryException` should be used when an allocation request would exceed the maximum heap size. `InvalidHeapAddressException` should be used when an address does not refer to a valid heap block. `HeapBoundsException` should be used when an index is outside an allocated block. `HeapUseAfterFreeException` should be used when a program attempts to free a block more than once or access a block after it has already been freed.

11 VirtualMachine Class

The VirtualMachine class is used for executing the given program. The VirtualMachine basically acts as the controller of this program. All operations need to go through the VirtualMachine class. It will contain the following data-fields (more can be added IF needed):

```
1 private RunTimeStack runTimeStack;
2 private Heap heap;
3 private Stack<Integer> returnAddress;
4 private Program program;
5 private int programCounter;
6 private boolean isRunning;
```

The VirtualMachine will contain many functions. These will not be listed. The reason for this is that you are expected to abstract the operations needed by the ByteCodes and create a clean and concise set of functions. Limit the amount of ByteCode logic that appears in the VirtualMachine. For example, if you are executing the ReadCode ByteCode, the process of reading is not done by the VirtualMachine; it is done by the ReadCode class, and then the ReadCode class requests the VirtualMachine to push the read value onto the RunTimeStack.

The same design applies to Heap operations. Heap ByteCodes should not directly access the Heap. They should request heap allocation, heap loading, heap storing, or heap freeing through clean VirtualMachine methods. The VirtualMachine should coordinate any values that need to move between the RunTimeStack and the Heap.

The returnAddress stack stores the ByteCode index (PC) that the VirtualMachine should execute when the current function exits. Each time a function is entered, the PC should be pushed onto the returnAddress stack. When a function exits, the PC should be restored to the value that is popped from the top of the returnAddress stack.

One important function in the VirtualMachine is executeProgram. A sample base function is given:

```
1 public void executeProgram(){
2     isRunning = true;
3
4     while(isRunning){
5         ByteCode code = program.getCode(programCounter);
6         code.execute(this);
7         programCounter++;
8     }
9 }
```

Note that we can easily add new ByteCodes without breaking the operation of the executeProgram function. We are using Dynamic Binding to achieve code flexibility, extensibility and readability. This loop should be very easy to read and understand.

12 VERBOSE mode

There is one special ByteCode that is used to determine whether the VirtualMachine should display information as it executes your program. This ByteCode is named VERBOSE. It will have 2 states, ON and OFF. No other form of VERBOSE will occur in this program or any given program. Note that your test files (.x.cod) may not contain these and you will need to add them yourself.

- VERBOSE ON is an interpreter command to turn on runtime VERBOSE mode. This will set an interpreter switch that will cause runTimeStack to display its contents in the correct format to the console AFTER the execution of EACH ByteCode. This switch or variable will be stored as a private data field in the VirtualMachine class.
- VERBOSE OFF will reset the switch to turn off VERBOSE mode.
- DO NOT display program state unless VERBOSE mode is turned ON.

Consider the following ByteCode program (ByteCodes marked with arrows are DISPLAYED):

```
GOTO start<<1>>
LABEL Read
RETURN
LABEL Write
LOAD 0 dummyFormal
WRITE
RETURN
LABEL start<<1>>
LIT 0 i
LIT 0 j
GOTO continue<<3>>
LABEL f<<2>>          <-- DISPLAYED
LIT 0 j              <-- DISPLAYED
LIT 0 k              <-- DISPLAYED
LOAD 0 i             <-- DISPLAYED
LOAD 0 j             <-- DISPLAYED
VERBOSE OFF          VERBOSE MODE IS TURNED OFF
BOP +
LOAD 2 k
BOP +
LIT 2
BOP +
RETURN f<<2>>
POP 2
LIT 0 GRATIS-RETURN-VALUE
RETURN f<<2>>
```

LABEL continue<<3>>	
VERBOSE ON	VERBOSE MODE IS TURNED ON
LIT 0 m	<-- DISPLAYED
LIT 3	<-- DISPLAYED
ARGS 1	<-- DISPLAYED
CALL f<<2>>	<-- DISPLAYED
VERBOSE ON	VERBOSE IS TURNED ON
STORE 2	<-- DISPLAYED
VERBOSE OFF	VERBOSE IS TURNED OFF
LOAD 1 j	
LOAD 2 m	
BOP +	
ARGS 1	
WRITE	
STORE 0 i	
POP 3	
HALT	

When VERBOSE mode is turned on you should print the following information **JUST AFTER** executing the current ByteCode:

- Print the ByteCode that was just executed.
- Print the runtime stack with spaces separating each activation frame. Values contained in one frame should be surrounded by [and].
- If VERBOSE mode is not on, then DO NOT print the ByteCode or display the contents of the runtime stack.

```

LIT 0 m      int m
      [0,0,0]
LIT 3
      [0,0,0,3]
ARGS 1
      [0,0,0] [3]
CALL f<<2>>  f(3)
      [0,0,0] [3]
LIT 0 j      int j
      [0,0,0] [3,0]
LIT 0 k      int k
      [0,0,0] [3,0,0]
LOAD 0 i <load i>
      [0,0,0] [3,0,0,3]
LOAD 1 j <load j>
      [0,0,0] [3,0,0,3,0]

```

```
...  
STORE 2 m    m = 2  
[0,0,5]
```

Note: The following shows the output if VERBOSE mode is on and 0 is at the top of the runtime stack.

```
RETURN f<<2>>  EXIT f:0
```

Note that 0 is returned from function f.

- If VERBOSE mode is turned on and we encounter an instruction such as WRITE, you may do the following. The value may be printed either before or after displaying the WRITE bytecode, for example:

```
LIT 3  
[0,0,0,3]  
3  
WRITE  
[0,0,0,3]
```

12.1 VERBOSE Implementation Notes

The VirtualMachine maintains the state of your running program, so it is a good place to have the VERBOSE flag. You CANNOT use a state variable in the ByteCode class.

The VERBOSE method should be a part of the RunTimeStack class (verboseDisplay). This method is called without any arguments. Therefore, there is no way to pass any information about the VirtualMachine or the ByteCode classes into RunTimeStack. As a result, you cannot do much outside logic inside RunTimeStack.verboseDisplay() except for displaying the state of the RunTimeStack itself. Also, **The ByteCode abstraction and its concrete classes should not call any verbose related methods in the VM. Nor should they ask the VirtualMachine if VERBOSE mode is on. The VirtualMachine knows all the information needed to trigger VERBOSE mode and display its contents. Each ByteCode SHOULD implement a method to support VERBOSE mode.**

It is impossible to determine the declared type of a variable by looking at the ByteCode file. To simplify matters you should assume whenever the interpreter encounters LIT 0 x (for example), that it is an int. When the interpreter is in VERBOSE mode, it is ok to represent the values as ints.

13 Coding Hints

13.1 Possible Order of Implementation

Below is an ordered list of how you may start this assignment. Note that this is not the only way. If you have a better way you may use your way.

1. **Read this PDF 3 or 4 times minimum.**
2. Think and design the ByteCode abstraction. This will help with completing other classes.
3. Implement and test the RunTimeStack class.
4. Implement and test the Heap class after the RunTimeStack class. The Heap should be tested independently before heap ByteCodes are completed.
5. Create all ByteCode classes listed in the table of ByteCodes listed above. The classes do not need to be fully implemented but having the method stubs (empty method bodies) is enough.
6. Implement the ByteCodeLoader class. Note as you implement the loadCodes function, you will need to start filling in the init functions for each of the ByteCode classes as well as adding helper functions to the Program class.
7. Implement the Program Class, specifically the resolveAddress function.
8. Implement the VirtualMachine Class.
9. While completing the implementation of the VirtualMachine class, you should also start to fill in the execute method of each of the ByteCode classes.

13.2 Other Coding Hints

Below are general coding hints and assumptions you may make about the project and the source files used.

- You will assume that the given ByteCode source programs (.cod files) you will use for testing are generated correctly and contain no errors.
- You will need to make sure that you protect the RunTimeStack from stack overflow or stack underflow errors. Also make sure no ByteCode can pop past any frame boundary.
- You will need to protect the Heap from invalid addresses, invalid indexes, use-after-free errors, double-free errors, and heap overflow errors.
- You should provide as much documentation as you can when implementing The Interpreter. Short, OBVIOUS functions do not need comments. However, you should comment each class describing its function and purpose. Take into consideration that your code is the first level of documentation. How you name things in your code matters. If you find yourself writing a lot of comments, then you may be either explaining

a complex algorithm, which is OK, or your code contains variable and method names that are not descriptive enough.

- **DO NOT** provide any methods that return any components contained WITHIN the VirtualMachine (this is the exact situation that will break encapsulation). You should request the VirtualMachine to perform operations on its components. This implies that the VirtualMachine owns the components and is free to change them as needed without breaking clients' code (for example, suppose I decided to change the name of the variable that holds my runtime stack. If your code had a reference to that variable, then your code would break. This is not an unusual situation; you can consider the names of methods in the Java libraries that have been marked deprecated).

The only downside is it might be a bit inefficient. Since I want to impress on everyone important software engineering issues, such as encapsulation benefits, I want to enforce the requirement that you **DO NOT BREAK** encapsulation.

Consider that the VirtualMachine calls the individual ByteCode's execute method and passes itself as a parameter. For the ByteCode to execute, it must invoke one or more runtime stack operations. It can do this by executing VirtualMachine.runTimeStack.pop(); however, this DOES break encapsulation. To avoid this, you will need to have a corresponding set of methods within the VirtualMachine that do nothing more than pass the call to the RunTimeStack. For example, a pop function can be implemented in the VirtualMachine in the following way

```
1 public int popRunStack() {  
2     return runTimeStack.pop();  
3 }
```

Then in a ByteCode's execute method, a pop can be executed as:

```
1 int temp = virtualMachine.popRunStack();
```

- Each ByteCode class should have fields for their specific arguments. The ByteCode abstraction and concrete classes **SHOULD NOT CONTAIN ANY FIELDS (instance variables) THAT ARE RELATED TO STORING ARGUMENTS AS A COLLECTION OR ARRAY. CONCRETE BYTECODES MUST STORE ARGUMENTS AS DATA-FIELDS AND THEY MUST BE THE CORRECT TYPE.** This is a design requirement.

It is easier to think in more general terms (i.e. plan for any number of arguments for a ByteCode). Note that the ByteCode abstraction should not be aware of the peculiarities of any ByteCode. That is, some ByteCodes might have zero arguments (HALT) or one argument, etc. Consider providing an init function with each ByteCode class. After constructing a ByteCode instance during the loadCodes phase, you can then call init passing in an ArrayList of String as arguments. Each ByteCode object will then interrogate the ArrayList and extract the needed arguments itself. The ByteCode abstraction should not record any arguments for any ByteCodes. Each ByteCode

concrete class will need instance variables for each of their arguments. There may be a need for additional instance variables as well.

- **The ByteCode abstraction and its concrete classes must be contained in the bytetimes package.**
- Any output produced by the WRITE ByteCode will be interspersed with the output from VERBOSE mode (if VERBOSE mode is turned on). In the WRITE action you should print one number per line. DO NOT print out something like:

Program Output: 2

Instead, only include the value and that is it. It should just be:

2

- There is no need to check for division by zero errors. Assume that you will not have a test case where this occurs.

14 Setup

14.1 Importing Project

When importing The Interpreter project, you will use the root of your repository as the source. **DO NOT USE** the interpreter folder as the root. This will cause the project to not work and in the end force you to change the file structure. As has been noted many times, **YOU ARE NOT ALLOWED TO MODIFY THE BASE STRUCTURE OF THIS PROJECT. DOING SO WILL CAUSE A POINT PENALTY TO BE APPLIED TO YOUR GRADE.**

When executing this project you will need to create the run configurations and set the command line arguments. The Interpreter is only able to handle the `.x.cod` files. The `.x` files are not going to be used for this project and are present only as a reference.

A sample picture of a run configuration is given below:

Note that the working directory and the Use Classpath of module values may not be the same. The purpose of the image is to show where the filename of the ByteCode source file goes.

15 Requirements

1. Implement ALL supported bytecode classes listed in the [Supported ByteCodes](#). Be sure to create the correct abstractions for the bytetimes. It is possible to have multiple abstractions within the set of bytecode classes.

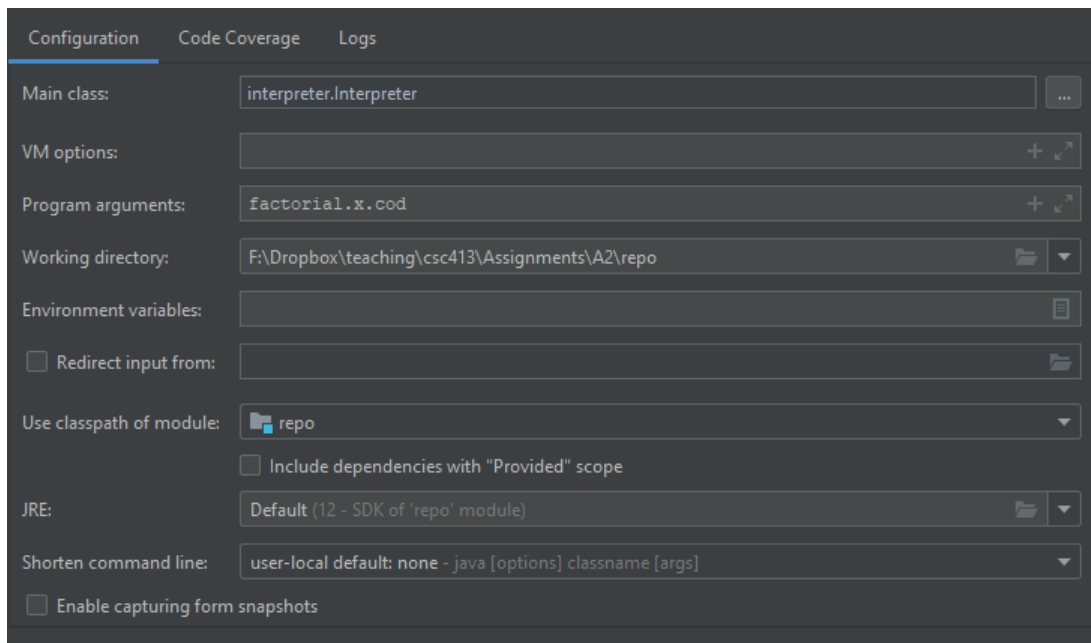


Figure 5: Sample picture of an IntelliJ run configuration for The Interpreter project.

2. Make sure to adhere to the design requirements listed in the above sections for each of the classes and the ByteCode abstraction.
3. Complete the implementation of the following classes:
 - a ByteClassLoader
 - b CodeTable
 - c Program
 - d RunTimeStack
 - e VirtualMachine
 - f Heap

The Interpreter class has already been implemented for you. Interpreter.java is the entry point to this project. All projects will be graded using this entry point. **The Interpreter class MUST NOT be changed. Doing so will cause your project to lose points.**

4. Make sure that all variables have their correct modifiers. Projects with variables using the incorrect modifiers will lose points.
5. Make sure not to break encapsulation. Projects that contain objects or classes trying to access members that it should not be allowed to will lose points. For example, if a ByteCode needs to access or write to the runtime stack, it should **NOT** be allowed to. It needs to request these operations from the virtual machine. Then the virtual machine will carry out the operation.

It would also be incorrect to make a method in the virtual machine for each bytecode. While this approach will work, this solution will produce a lot of duplicate code. Points will be deducted for this type of solution. Do your best to understand the operations of each ByteCode and how they manipulate the data structures the VirtualMachine maintains. You will be able to see that some ByteCodes operate on these data structures in a very similar way. Basically, you are being asked to code to the virtual machine and not to the bytecodes.

6. The basic structure of the interpreter folder and its contents cannot be changed. You may add sub-folders to the bytecodes folder and the package-specific exceptions folders described in this document. **10 POINTS WILL BE DEDUCTED if the structure of the project is changed.**

16 Submission

1. All completed source code needs to be submitted to your repository created with the link on Canvas. Use the following commands to commit your code. Please note you must be in the repository folder to run these commands.

```
1 $ git add .
2 $ git commit -m "message"
3 $ git push origin main
```

2. Complete the required documentation as outlined in the documentation guidelines. This is stored in the documentation folder. Convert the completed document to a PDF file and save in the documentation folder. **PDF FILES ARE REQUIRED. 5 POINTS WILL BE DEDUCTED FOR NOT SUBMITTING A PDF**